BRAILLEBRIDGE

Technical Documentation

Architecture, chord mathematics, firmware, protocol, build and operating guide for the BrailleBridge keyboard.

Document	Technical documentation, version 1.0, October 2026
Author	Samin Yeasar
Email	Sheditzofficial918@gmail.com
Portfolio	solez.vercel.app
GitHub	github.com/Solez-ai/braille.bridge
Official website	braile-bridge.vercel.app
Documentation	braille.mintlify.site
Covers	Firmware v4 (braile.ino with vendored BBKeyboard), Teacher Software, serial and BLE protocol, hardware

1. System overview

BrailleBridge turns six Braille dot keys, a Space key and a Shift key into text. An ESP32 reads the keys, merges near-simultaneous presses into a 6-bit chord, looks the chord up in a dictionary for the active language, and sends the result on three outputs: a Bluetooth LE keyboard (HID), a Bluetooth LE Nordic UART stream, and USB serial. All translation happens on the device, so every host sees finished characters.

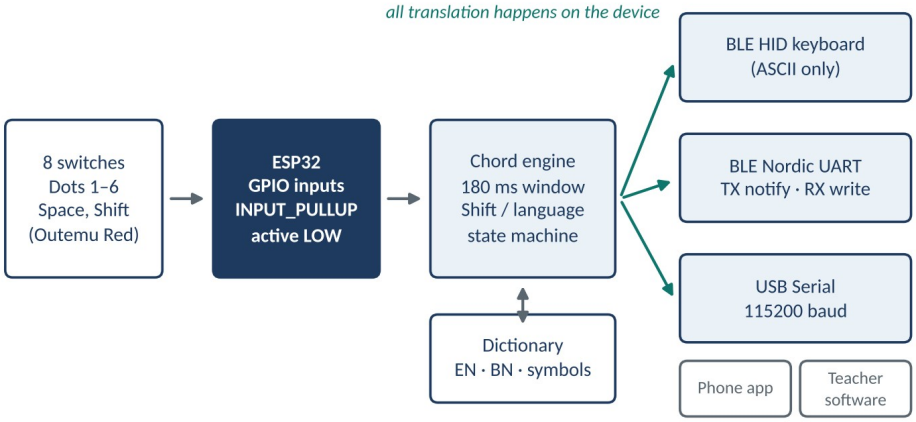


Figure 1. Signal path from switches to hosts. Translation happens once, on the ESP32.

2. Hardware

2.1 Bill of materials

Component	Quantity	Unit note	Cost (USD)
ESP32 Dev Module	1		5.00
Outemu Red switch	8	6 dots, Space, Shift	3.00
Keycaps (3D-printed)	8		1.00
LED (3 mm) and resistor	2 + 2	language indicators	0.30
Perfboard	1		0.80
PLA filament (enclosure)	30 g		1.20
M3 screws, wire, solder	—		0.70
Total (prototype parts cost)			**about 12.00**

Figures as published in the project repository [8]. They exclude tools, labour and shipping.

2.2 Pin assignment and electrical notes

Function	GPIO	Cell position	Bit	Notes
Dot 1	14	top-left	0	weight 1
Dot 2	12	middle-left	1	weight 2; boot-strapping pin, see note
Dot 3	13	bottom-left	2	weight 4
Dot 4	27	top-right	3	weight 8
Dot 5	26	middle-right	4	weight 16
Dot 6	25	bottom-right	5	weight 32
Space	33	dedicated key	—	acts only when no dot is held

Function	GPIO	Cell position	Bit	Notes
Shift	32	modifier key	—	hold = modifier, tap = Backspace
Red LED	18	English indicator	—	optional indicator
Green LED	19	Bangla indicator	—	optional indicator

- **Wiring.** Each switch connects one pin to a shared ground bus. The firmware sets INPUT_PULLUP, so an unpressed key reads HIGH and a pressed key reads LOW. No external resistors are needed.
- **Strapping pin.** GPIO12 selects the flash voltage during boot on the classic ESP32. The Dot 2 switch pulls it to ground, which is the safe default level, so the design is compatible. Do not add an external pull-up to this line.
- **Debounce.** There is no hardware debounce on the dot keys. Software tolerance comes from the 180 ms chord window and the 50 ms lockout after each commit (section 4).
- **Enclosure.** Print both shells in PLA at 0.2 mm layers. The MX-compatible cutouts let switches snap in without glue or solder, and the shells close with M3 screws.

3. Chord encoding and mathematics

3.1 Bit encoding

A Braille cell has six dots, numbered 1 to 3 down the left column and 4 to 6 down the right. Dot n owns bit n−1 of an integer, so a chord is a sum of powers of two and every chord is a unique value from 1 to 63. Zero means no key is pressed.

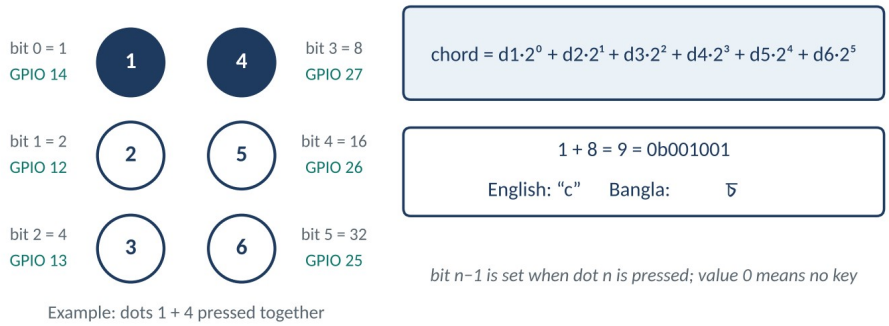


Figure 2. Dot layout, bit weights, GPIO pins and a worked example (dots 1 and 4).

```
// dots 1 and 4 held together within the chord window
currentChord |= (1 << 0);   // dot 1  -> 0b000001
currentChord |= (1 << 3);   // dot 4  -> 0b001000
// result 0b001001 = 9 -> lookup -> "c" (English) or Bangla ৎ
```

3.2 Counting the code space

Six binary keys give $2^6 = 64$ states, of which 63 are non-empty chords. The number of chords that use exactly k dots is the binomial coefficient C(6, k) = 6, 15, 20, 15, 6, 1 for k = 1 to 6. Because Shift and the language flag each select a different table, the device exposes four layers of up to 63 slots (English, Bangla, Shift-English, Shift-Bangla), or up to 252 addressable outputs from eight keys.

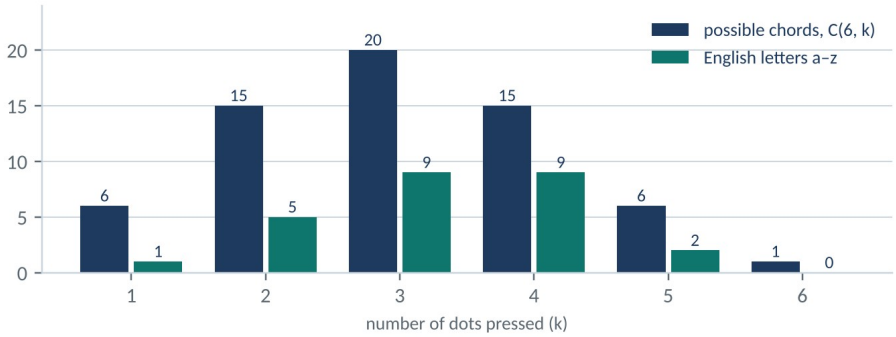


Figure 3. Possible chords by number of dots (navy) against the English letters that use them (teal).

3.3 English table (Grade 1 Braille)

Letter	Dots	Binary	Hex	Letter	Dots	Binary	Hex
a	1	0b000001	0x01	n	1,3,4,5	0b011101	0x1D
b	1,2	0b000011	0x03	o	1,3,5	0b010101	0x15
c	1,4	0b001001	0x09	p	1,2,3,4	0b001111	0x0F
d	1,4,5	0b011001	0x19	q	1,2,3,4,5	0b011111	0x1F
e	1,5	0b010001	0x11	r	1,2,3,5	0b010111	0x17
f	1,2,4	0b001011	0x0B	s	2,3,4	0b001110	0x0E
g	1,2,4,5	0b011011	0x1B	t	2,3,4,5	0b011110	0x1E
h	1,2,5	0b010011	0x13	u	1,3,6	0b100101	0x25
i	2,4	0b001010	0x0A	v	1,2,3,6	0b100111	0x27
j	2,4,5	0b011010	0x1A	w	2,4,5,6	0b111010	0x3A
k	1,3	0b000101	0x05	x	1,3,4,6	0b101101	0x2D
l	1,2,3	0b000111	0x07	y	1,3,4,5,6	0b111101	0x3D
m	1,3,4	0b001101	0x0D	z	1,3,5,6	0b110101	0x35

Grade 1 Braille values.

3.4 Structure of the alphabet

The English letters follow a rule that makes the table easy to verify. Letters a to j use dots from the top four positions (1, 2, 4, 5). The second decade, k to t, adds dot 3 to the first, so chord(k to t) = chord(a to j) OR 0b000100. The third decade, u, v, x, y and z, adds dots 3 and 6 to a to e, so OR 0b100100. The letter w is the exception, equal to j plus dot 6 (0b111010). A script asserted all three rules against the 26-letter table, and they hold.

```
def chord(dots): return sum(1 << (d - 1) for d in dots)
assert chord([1, 4]) == 0b001001          # c
assert chord([2, 4, 5, 6]) == 0x3A        # w = j | dot 6
assert all(K[i] == K[i-10] | 0b000100 for i in range(10, 20))  # k..t
```

3.5 Confusability: Hamming distance

Two chords that differ by one dot have Hamming distance 1. A single missed or extra finger then turns one letter into another. Among the 26 English letters there are 51 such pairs. The most exposed letters are f, g, k, l, m, n, o, p and q, each with five one-dot neighbours; the letter w has only one (j). This is why the firmware waits for the full chord window before committing, rather than acting on the first key that lands.

3.6 Timing model and throughput bound

Let W be the chord window and L the lockout after each commit. A chord is read correctly when the skew S between the first and last finger is smaller than W, leaving a margin W − S. The firmware ceiling on chord rate is 1 / (W + L), because a held chord commits at W and the next one is ignored for L. The table shows the trade-off; the shipped value is W = 180 ms, L = 50 ms.

Window W (ms)	Cycle W + L (ms)	Chords per second	Characters per minute
120	170	5.9	353
180 (shipped)	**230**	**4.3**	**261**
250	300	3.3	200

At 5 characters per word the shipped setting caps the firmware at about 52 words per minute. A chord whose keys all release sooner commits at release, so this is an upper bound set by the device, not by typists.

3.7 Shift layer and Bangla vowel signs

When Shift is held, the firmware checks tables in a fixed order: digits, then maths and punctuation, then English capitals, and in Bangla mode the vowel-sign table and finally the ordinary Bangla cell. Shift plus a to j produces the digits 1 to 9 and 0.

Output	Dots	Binary	Output	Dots	Binary
+	2,3,5	0b010110	×	2,3,6	0b100110
- (hyphen)	3,6	0b100100	=	2,3,5,6	0b110110
÷	3,4	0b001100	,	2	0b000010
;	2,3	0b000110	:	2,5	0b010010
.	2,5,6	0b110010			

Symbols are language-independent and need Shift. Plain ASCII symbols also go to the Bluetooth keyboard.

Vowel	Chord dots	Sign (Shift)	Vowel	Chord dots	Sign (Shift)
আ	3,4,5	া	এ	1,5	ে
ই	2,4	ি	ঐ	3,4	ৈ
ঈ	3,5	ী	ও	1,3,5	ো
উ	1,3,6	ু	ঔ	2,4,6	ৌ
ঊ	1,2,5,6	ূ	অ	1	none

Bangla dependent vowel signs (kar). The vowel অ has no sign, so Shift plus dot 1 falls through to the ordinary letter.

3.8 Bangla cells in the firmware dictionary

The dictionary holds 52 Bangla entries, following the Bharati Braille family. Several cells are shared with English letters. Five entries carry the dotted circle because they are combining marks: halant (dot 4), anusvara (dots 5, 6), candrabindu (dot 5), and the convenience vowel signs for i and long u.

Cell	Dots	Binary	Cell	Dots	Binary	Cell	Dots	Binary
অ	1	0b000001	র	1,2,3,5	0b010111	ঙ	3,4,6	0b101100
ব	1,2	0b000011	ভ	4,5	0b011000	য	1,3,4,6	0b101101
ক	1,3	0b000101	দ	1,4,5	0b011001	ধ	2,3,4,6	0b101110
ল	1,2,3	0b000111	জ	2,4,5	0b011010	ষ	1,2,3,4,6	0b101111

Cell	Dots	Binary	Cell	Dots	Binary	Cell	Dots	Binary
্	4	0b001000	গ	1,2,4,5	0b011011	ং	5,6	0b110000
চ	1,4	0b001001	আ	3,4,5	0b011100	শ	1,5,6	0b110001
ই	2,4	0b001010	ন	1,3,4,5	0b011101	ূ	2,5,6	0b110010
ঐ	3,4	0b001100	ত	2,3,4,5	0b011110	ঊ	1,2,5,6	0b110011
ম	1,3,4	0b001101	ক্ষ	1,2,3,4,5	0b011111	ঝ	3,5,6	0b110100
স	2,3,4	0b001110	ছ	1,6	0b100001	ড়	1,3,5,6	0b110101
প	1,2,3,4	0b001111	স	2,6	0b100010	ঠ	2,3,5,6	0b110110
ঁ	5	0b010000	ঘ	1,2,6	0b100011	থ	1,4,5,6	0b111001
এ	1,5	0b010001	উ	1,3,6	0b100101	ণ	3,4,5,6	0b111100
ঞ	2,5	0b010010	ি	2,3,6	0b100110	য	1,3,4,5,6	0b111101
হ	1,2,5	0b010011	ৱ	1,2,3,6	0b100111	ট	2,3,4,5,6	0b111110
ঈ	3,5	0b010100	খ	4,6	0b101000	ঢ	1,2,3,4,5,6	0b111111
ও	1,3,5	0b010101	ঔ	2,4,6	0b101010			
ফ	2,3,5	0b010110	ড	1,2,4,6	0b101011			

4. Firmware architecture

4.1 The chord engine

The main loop polls all eight keys every pass. The first dot down starts a chord and records the time. While the chord is active, each pass ORs the currently pressed dots into the chord value. The chord commits when every dot is released or when 180 ms have passed since the first press, whichever comes first. A special case holds back commit for the exact mask of dots 4, 5 and 6, so a half-second hold can toggle the language instead. After a commit or a toggle the engine waits 50 ms and returns to idle; Shift tap handling runs alongside it (section 4.4).

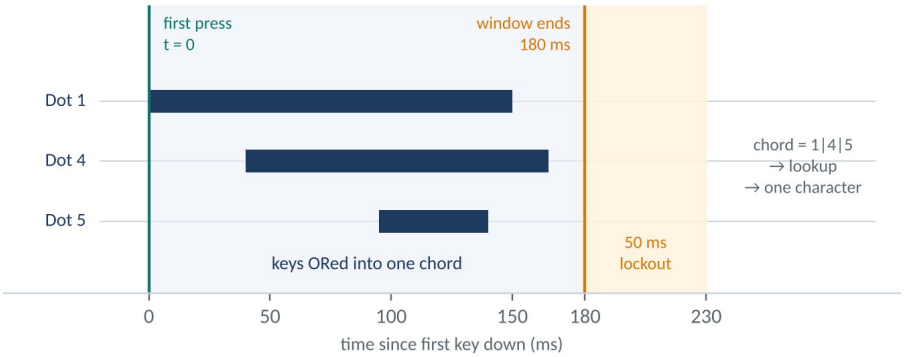


Figure 4. Chord window: keys landing within 180 ms are ORed into one chord, followed by a 50 ms lockout.

```
if (anyDot) {
  if (!chordActive) { chordActive = true; pressStartTime = millis(); currentChord = 0; }
  if (d1) currentChord |= (1 << 0);   // ... d2..d6 likewise
}
if (chordActive) {
  unsigned long elapsed = millis() - pressStartTime;
  if (currentChord == 0b111000 && elapsed >= 500 && !allReleased) { toggleLanguage(); return; }
  if (allReleased || elapsed > 180) { processChord(currentChord); chordActive = false; delay(50); }
}
```

4.2 Timing constants

Constant	Value	Purpose
Chord window	180 ms	Keys within this time after the first press form one chord
Language toggle hold	500 ms	Dots 4, 5 and 6 held together switch English and Bangla
Post-commit lockout	50 ms	Blocks bounce and accidental double reads after each character
Space debounce	200 ms	Space types once, then waits for release
Shift tap, minimum	25 ms	Shorter presses are treated as bounce
Shift tap, maximum	1500 ms	Longer holds are ignored so a pocketed device cannot delete text
Backspace cooldown	100 ms	Stops release bounce from firing twice
HID Backspace pulse	8 ms	Key down to key up on the Bluetooth keyboard
Serial speed	115200 baud	USB serial; mirrored over the NUS stream
BLE MTU	247 bytes	Large enough for any single character or control line

4.3 Dictionary and lookup

Translations live in an array of BrailleEntry records holding a chord, an English string and a Bangla string. A null string means the chord exists in the table but has no meaning in that language, and the press is ignored silently. Lookup is a linear scan; with about 52 entries this takes microseconds on a 240 MHz core, and the table occupies well under 1 KB, so a hash is not needed. Adding a language means adding a column or a second table, not changing control flow.

```
struct BrailleEntry { uint8_t chord; const char* english; const char* bangla; };
// processChord(), Shift held:  digits -> math/punctuation -> EN capital -> BN kar -> BN cell -> ignore
```

4.4 Shift tap and Backspace

Shift is a modifier whenever dots are involved. A clean tap with no chord sends one Backspace on both outputs: a real HID Backspace over Bluetooth and a SYSTEM:BKSP line on the serial and NUS streams. Four guards prevent misfires: a dot held during the tap vetoes it, a press under 25 ms is ignored as bounce, a hold over 1.5 s is ignored, and a 100 ms cooldown follows each delete.

4.5 The ASCII rule for the keyboard channel

HID key codes cover ASCII only. Pushing multi-byte UTF-8 such as Bangla through the keyboard stack was found to wedge the ESP32, so sendChar() checks every byte and sends to the HID keyboard only when all bytes are below 0x80. Every character, ASCII or not, goes to the serial and NUS streams. Bangla therefore travels to the Teacher Software through these streams.

5. Communication layer

5.1 Channels

Channel	Details	Carries
BLE HID keyboard	Service 0x1812, appearance 0x03C1, name “BrailleBridge”, secure connections with bonding; T-vK BleKeyboard report map	ASCII characters and Backspace
BLE Nordic UART	Service 6E400001-B5A3-F393-E0A9-E50E24DCCA9E; notify characteristic … 0002, write characteristic … 0003; MTU 247	Full character stream, LANG:*, SYSTEM:BKSP; receives LANG:en and LANG:bn
USB serial	115200 baud, 8N1	Identical stream to NUS

5.2 Serial protocol

Characters are sent bare, with no framing. Control lines end with a newline. The host separates them by splitting on newlines and treating anything that matches a control pattern as a command.

Message	Meaning
LANG:en, LANG:bn	Language switched; sent on toggle and on a command from the host
SYSTEM:BKSP	Shift tap; the host deletes the last character
SYSTEM: …	Banner and status lines, never rendered as text
Invalid	Rejected control command from the host
any other text	Translated character data

At power-up the ESP32 ROM prints boot text at 115200 baud. Hosts drop lines beginning ets , rst:0x, boot:0x, configSPIWP, clk_drv:, q_drv:, d_drv:, cs0_drv:, hd_drv:, wp_drv:, mode:DIO, load:0x and entry 0x. After ets the filter stays on until entry 0x or a SYSTEM: line arrives.

5.3 Teacher Software

The Teacher Software is plain HTML, CSS and JavaScript that talks to each device through the Web Serial API in Chrome or Edge. For every student it opens a port at the chosen baud rate, decodes bytes incrementally as UTF-8 so a Bangla character split across reads is never corrupted, and runs a dedicated read loop that feeds each character to that student's feed.

```
const port = await navigator.serial.requestPort();
await port.open({ baudRate: student.baud });
port.readable.pipeTo(decoder.writable);          // TextDecoderStream
for (;;) { const { value, done } = await reader.read(); if (done) break;
          for (const ch of value) handleStudentChar(student, ch); }
```

- **Roster and storage.** A persistent multi-student roster with per-student port and baud rate is kept in the browser's local storage.
- **Views.** Live feeds with language detection from the LANG:* lines, an exam mode that shows every student side by side, and an expanded view for one student.
- **Records.** Every character is timestamped [HH:MM:SS] and exports as a text file. A 500 ms refresh rebuilds each feed from stored data.

6. Build and operating guide

6.1 Build steps

1. **Print.** Print TOP_SHELL.stl and BOTTOM_SHELL.stl in PLA at 0.2 mm layers, plus eight keycaps.
2. **Wire.** Snap eight switches into the cutouts. Solder one leg of each to the shared ground bus and the other to its GPIO pin from section 2.2.
3. **Flash.** Install the Arduino IDE and the ESP32 core, open braile/braile.ino with the vendored BBKeyboard files beside it, select ESP32 Dev Module and upload. No libraries are needed.
4. **Power on.** Supply 5 V over USB. The board advertises over Bluetooth LE as “BrailleBridge”.
5. **Connect the teacher.** Open braille-display/index.html in Chrome or Edge, add a student, connect the USB port at 115200 baud, then type a chord to confirm.

6.2 Operating cheat sheet

Action	Keys	Result
Type a letter	Press its chord within 180 ms	One character
Space	Space key, no dots held	Space
Digit	Hold Shift, press a to j chord	1 to 9 and 0
Capital letter	Hold Shift, press a letter chord (k to z at present)	Upper case
Symbol	Hold Shift, press a symbol chord	Punctuation or maths
Bangla vowel sign	In Bangla mode, hold Shift, press a vowel chord	Dependent sign (kar)
Backspace	Tap Shift alone	Deletes one character
Switch language	Hold dots 4, 5 and 6 for 500 ms	LANG:en or LANG:bn

6.3 Troubleshooting

Symptom	Likely cause	Action
Teacher Software shows no device	Not powered, out of range, or not connected	Check power, move closer, reconnect
Teacher Software sees no port	Unsupported browser, or port busy	Use Chrome or Edge; close other serial tools; set 115200
Garbage at power-up	ESP32 ROM boot text	Handled by the boot filter; reload if it persists
A chord does nothing	No entry in the active language	Check the tables; unknown chords are ignored
A letter repeats	Keys held after commit	Release all keys fully between chords

7. Verification and future work

7.1 What has been verified

The firmware has been through four on-hardware revisions, documented in the repository: a hand-built GATT layer was replaced, and v4 returned to the proven T-vK BleKeyboard code with the Nordic UART Service added. The chord tables in this document were checked against the source, and the 26-letter structure rules in section 3.4 were verified programmatically.

7.2 Iterations in progress

The following work is under way to make typing faster, more forgiving and more comfortable:

- **Autocorrect.** Correction of mistyped chords and words, informed by the Hamming-distance analysis in section 3.5.
- **Better typing on the keys.** Refining chord recognition so fast, natural typing is read accurately.
- **Faster response timings.** Tuning the chord window and post-commit lockout in section 4.2 against real typing speeds.
- **New input methods.** Exploring clever ways to enter common letters, words and actions faster than a plain chord.
- **Ergonomic layout.** Redesigning the key layout and enclosure for comfort during long sessions.
- **Roadmap items.** Mathematical Braille, battery power with USB-C charging, audio and vibration feedback, secure examination mode, document export and multi-school deployment [8].

Sources: the BrailleBridge repository, documentation and website [8]: github.com/Solez-ai/braille.bridge, braille.mintlify.site, braile-bridge.vercel.app.